

COLLEGIUM WITELONA
Uczelnia Państwowa



Wydział Nauk Technicznych i Ekonomicznych
Kierunek Informatyka

Wydatkomat – aplikacja do kontrolowania grupowych wydatków

Sebastian Górski
Jakub Grzymisławski
Łukasz Szenkiel
Rafał Wilczewski

Legnica, 2026 rok

Spis treści

1	Wstęp	3
1.1	Cel projektu	3
1.2	Zakres projektu	4
2	Opis funkcjonalny systemu	5
2.1	Uwierzytelnianie i autoryzacja użytkowników	5
2.2	Zarządzanie wydatkami	5
2.3	Znajomi	6
2.4	Opłaty	6
2.5	Profil	6
2.6	Powiadomienia	7
2.7	Zarządzanie użytkownikami	7
2.8	Statystyki	7
2.9	Dokumentacja API	7
3	Opis technologiczny	8
3.1	API	8
3.1.1	Architektura aplikacji	8
3.1.2	Technologie	8
3.1.3	Baza danych	9
3.1.4	Bezpieczeństwo	9
3.1.5	Testy	9
3.2	Aplikacja webowa	10
3.3	Aplikacja mobilna	10
3.4	Aplikacja desktopowa	10
4	Podział obowiązków w zespole	11
4.1	Backend	11
4.2	Klient web	11
4.3	Klient mobile	12
4.4	Klient desktop	12

4.5	Bezpieczeństwo systemu	12
4.6	Baza danych	13
4.7	Deployment i infrastruktura	13
4.8	Testy	13
5	Instrukcja uruchomienia systemu	14
5.1	Backend (REST API)	14
5.2	Aplikacja webowa	14
5.3	Aplikacja mobilna	15
5.4	Aplikacja desktopowa	16
6	Wnioski projektowe	18
7	Źródła	19

1 Wstęp

Aplikacja Wydatkomat została stworzona w celu uproszczenia zarządzania wspólnymi wydatkami pomiędzy znajomymi. System umożliwia kontrolowanie kosztów, tworzenie grup użytkowników, dodawanie wydatków oraz generowanie rozliczeń pomiędzy członkami grupy. Dzięki temu użytkownicy mogą w prosty sposób monitorować swoje zobowiązania finansowe oraz historię wydatków.

Projekt został zrealizowany jako aplikacja wieloplatformowa, dostępna z różnych urządzeń i systemów operacyjnych. System składa się z centralnego backendu udostępniającego REST API oraz klientów korzystających z tego API. Przygotowano:

- aplikację webową stworzoną przy użyciu technologii React,
- aplikację mobilną opartą o React Native,
- aplikację desktopową napisaną w języku Python.

Backend aplikacji odpowiada za przetwarzanie danych, autoryzację użytkowników, logikę biznesową oraz komunikację z bazą danych. Komunikacja pomiędzy klientami a serwerem odbywa się przy użyciu protokołu HTTP oraz architektury REST.

Dodatkowym elementem systemu są statystyki wydatków, które pozwalają użytkownikom analizować swoje finanse oraz lepiej kontrolować budżet.

1.1 Cel projektu

Celem projektu było zaprojektowanie, implementacja, przetestowanie oraz wdrożenie wieloplatformowego systemu do zarządzania wspólnymi wydatkami. Projekt miał umożliwiać korzystanie z aplikacji na różnych urządzeniach przy zachowaniu wspólnego backendu oraz spójnej logiki działania systemu.

Istotnym założeniem projektu było również przygotowanie bezpiecznego i skalowalnego REST API, które może być wykorzystywane przez różne klienty aplikacji.

Aplikacja mobilna oraz desktopowa powinna umożliwiać również pracę w trybie offline, z późniejszą synchronizacją danych po ponownym połączeniu do sieci.

1.2 Zakres projektu

Zakres projektu obejmuje implementację systemu umożliwiającego:

- rejestrację i uwierzytelnianie użytkowników,
- zarządzanie swoim kontem użytkownika,
- zarządzanie znajomymi i uczestnikami wydatków,
- dodawanie i edytowanie wydatków,
- podział kosztów pomiędzy użytkownikami,
- przeglądanie historii wydatków,
- generowanie statystyk finansowych,
- zarządzanie użytkownikami jako adminstrator,
- otrzymywanie powiadomień wybranymi przez siebie kanałami.

Projekt obejmuje również przygotowanie dokumentacji REST API zgodnej ze standardem OpenAPI oraz wdrożenie aplikacji w środowisku zdalnym.

2 Opis funkcjonalny systemu

Poniżej przedstawione zostały najważniejsze funkcje zapewniane przez aplikację Wydatkomat. Należy mieć na uwadze, że aplikacja desktopowa została stworzona głównie jako panel administratora, w związku z czym nie zapewnia wszystkich funkcji. Z drugiej strony aplikacja mobilna i webowa nie zapewniają możliwości administrowania użytkownikami.

2.1 Uwierzytelnianie i autoryzacja użytkowników

Aby korzystać z aplikacji należy się zarejestrować. Jest to pierwsza funkcjonalność, która jest obligatoryjna, aby w pełni korzystać z systemu. Użytkownik może zarejestrować się formularzem zaimplementowanym w klientach lub poprzez protokół OAuth2 (Google, Facebook). Obie metody wiążą się z założeniem konta w systemie.

Po rejestracji należy się zalogować, sposoby logowania są analogiczne do sposobów rejestracji.

Użytkownik może połączyć swoje lokalnie utworzone konto z wieloma mediami społecznościowymi za pomocą protokołu OAuth2, co pozwoli na logowanie się obiema metodami.

Z drugiej strony użytkownik, który założył konto logując się przez OAuth2, ma możliwość utworzenia hasła, aby móc logować się formularzem z poziomu aplikacji.

Po poprawnym zalogowaniu przypisane zostają do użytkownika dwa tokeny: JWT oraz refresh token. Token dostępowy JWT jest ważny przez okres 1 godziny, po tym czasie sesja zostaje unieważniona. Aby ponownie móc korzystać z systemu należy się zalogować lub wykorzystać refresh token, który jest ważny przez tydzień.

Dodatkowo system umożliwia włączenie logowania dwuetapowego przy użyciu aplikacji mobilnej Google Authenticator. Aby to zrobić należy uruchomić opcję z poziomu profilu. Po włączeniu tej funkcji i poprawnym połączeniu aplikacji z Wydatkomatem, przy kolejnych próbach logowania należy przejść również drugi etap logowania – zweryfikowanie się kodem z aplikacji Google Authenticator.

2.2 Zarządzanie wydatkami

Główną funkcją aplikacji jest tworzenie nowych wydatków i edycja już istniejących. Użytkownik może utworzyć wydatek, dodając do niego swoich znajomych. Wydatek może mieć tytuł, opis, kwotę i wielu uczestników. Tylko osoba tworząca wydatek może go edytować. Przy edycji

wydatków bez problemu można zmieniać tytuł i opis. Można również zmieniać kwotę i uczestników, lecz w tym przypadku system waliduje aktualnie spłacone kwoty i może zablokować taką edycję. Jeśli edycja zostanie dozwolona, następuje rekalkulacja należnych kwot.

W aktualnej wersji systemu dostępne jest jedynie dzielenie wydatków po równo, co oznacza, że w przypadku utworzenia wydatku na 500 złotych i dodaniu 4 znajomych, kwota zostanie podzielona na 125 zł / osobę. Właściciel wydatku nie może być jego uczestnikiem.

Wydatki oraz ich uczestników można również usuwać.

2.3 Znajomi

Uczestnikami wydatków mogą być jedynie znajomi osoby, która go utworzyła. Każdy użytkownik może dodać znajomego po wyszukaniu go z listy aktywnych użytkowników systemu.

Po wysłaniu zaproszenia do znajomych osoba zapraszana otrzymuje stosowne powiadomienie, które może zaakceptować lub odrzucić.

Użytkownicy mogą sprawdzić oczekujące zaproszenia, zarówno wysłane jak i odebrane. Mogą również zobaczyć listę swoich znajomych.

Istnieje możliwość zakończenia znajomości z innym użytkownikiem. Znajomości są dwukierunkowe, co oznacza, że po akceptacji zaproszenia do znajomych obie strony widzą siebie na liście znajomych i mogą tworzyć między sobą wydatki.

2.4 Opłaty

Uczestnicy wydatku widzą sumaryczną kwotę wydatku, swoją część do spłaty oraz część, którą już spłacili.

Wydatek można opłacać stopniowo, czyli do jednego wydatku pojedynczy użytkownik może stworzyć wiele spłat.

Istnieje możliwość usunięcia stworzonej przez siebie płatności.

2.5 Profil

Zalogowany użytkownik może podejrzeć i edytować swój profil.

Z poziomu profilu można zarządzać swoim kontem, uwzględniając w to:

- ustawienie hasła lokalnego (jeśli takie jeszcze nie istnieje, dotyczy użytkowników OAuth2),
- połączenie kont społecznościowych,
- zmianę hasła,
- włączenie/wyłączenie logowania dwuetapowego,
- aktualizację swoich danych osobowych,
- usunięcie konta.

2.6 Powiadomienia

Każdy użytkownik może sterować kanałami powiadomień. W aplikacji na obecnym etapie dostępne są dwa kanały – email i websocket (powiadomienia realtime w aplikacji).

Powiadomienia dotyczą np.:

- zaproszenia do znajomych,
- nowego wydatku,
- nieudanej próby logowania.

Powiadomienia związane z bezpieczeństwem są na stałe przypisane do kanału email.

Preferencje dotyczące powiadomień są edytowalne. Powiadomienia można oznaczyć jako przeczytane w przypadku powiadomień websocket.

2.7 Zarządzanie użytkownikami

Jako administrator istnieje możliwość zarządzania użytkownikami.

Administrator widzi pełną listę aktywnych użytkowników, może dezaktywować użytkownika, zresetować lub zmienić jego hasło oraz utworzyć nowego użytkownika.

2.8 Statystyki

System umożliwia podgląd okresowych statystyk zarówno dotyczących wydatków jak i spłat. Umożliwia to lepsze zrozumienie i kontrolowanie budżetu.

Statystyki uwzględniają:

- całkowitą liczbę wydatków/spłat,
- całkowitą kwotę wydatków/spłat,
- średnią kwotę wydatków/spłat.

2.9 Dokumentacja API

Poza typowymi funkcjonalnościami systemu udostępniona publicznie jest również dokumentacja API w standardzie OpenAPI, które można znaleźć pod adresem URL:

<https://sgorski00.github.io/expense-splitter-api/>

Można w niej znaleźć pełną dokumentację wszystkich funkcjonalności i endpointów końcowych.

3 Opis technologiczny

3.1 API

3.1.1 Architektura aplikacji

Backend aplikacji Wydatkomat został zaimplementowany w architekturze REST API. System wykorzystuje model klient–serwer, w którym aplikacje webowa, mobilna oraz desktopowa komunikują się z centralnym backendem za pomocą protokołu HTTP.

Aplikacja została zaprojektowana zgodnie z architekturą warstwową. Logika systemu została podzielona na warstwy odpowiedzialne za:

- obsługę żądań HTTP,
- logikę biznesową,
- komunikację z bazą danych,
- bezpieczeństwo i autoryzację użytkowników.

REST API udostępnia endpointy umożliwiające zarządzanie użytkownikami, wydatkami, płatnościami, znajomymi, powiadomieniami oraz statystykami. Dokumentacja endpointów została przygotowana zgodnie ze standardem OpenAPI i udostępniona przy użyciu Swagger UI.

3.1.2 Technologie

Backend aplikacji został napisany w języku Java w wersji 25 z wykorzystaniem frameworka Spring Boot 4. Do implementacji poszczególnych elementów systemu wykorzystano między innymi:

- Spring Web – obsługa REST API,
- Spring Security – uwierzytelnianie i autoryzacja,
- Spring Data JPA – komunikacja z bazą danych,
- Hibernate – mapowanie obiektowo-relacyjne,
- PostgreSQL – relacyjna baza danych,

-
- JWT – obsługa tokenów autoryzacyjnych,
 - OAuth2 – logowanie zewnętrzne,
 - Docker oraz Docker Compose – konteneryzacja i wdrożenie aplikacji.

Do dokumentowania API wykorzystano standard OpenAPI oraz narzędzie Swagger UI.

3.1.3 Baza danych

System wykorzystuje relacyjną bazę danych PostgreSQL. Dane aplikacji przechowywane są w postaci encji powiązanych relacjami.

Komunikacja z bazą danych realizowana jest przy użyciu Spring Data JPA oraz Hibernate. Zastosowanie ORM pozwoliło ograniczyć liczbę bezpośrednich zapytań SQL oraz uprościć zarządzanie danymi.

Struktura bazy danych jest wersjonowana przy użyciu narzędzie Flyway.

3.1.4 Bezpieczeństwo

Backend aplikacji został zabezpieczony przy użyciu mechanizmów dostępnych w Spring Security. System wykorzystuje uwierzytelnianie JWT, dzięki któremu możliwe jest bezstanowe zarządzanie sesją użytkownika.

Hasła użytkowników przechowywane są w postaci skrótów generowanych przy użyciu algorytmu Argon2. System wspiera również logowanie OAuth2 oraz uwierzytelnianie dwuetapowe z wykorzystaniem aplikacji Google Authenticator.

Komunikacja pomiędzy klientem a serwerem odbywa się wyłącznie przy użyciu protokołu HTTPS zabezpieczonego certyfikatem TLS 1.3. Dodatkowo system wykorzystuje mechanizmy ochrony przed CSRF oraz SQL Injection.

3.1.5 Testy

W projekcie zaimplementowano testy jednostkowe oraz integracyjne backendu. Testy wykorzystywane są do weryfikacji poprawności logiki biznesowej, autoryzacji użytkowników oraz działania endpointów REST API.

Do testów integracyjnych wykorzystano środowisko uruchamiane w kontenerach Docker przy użyciu narzędzia Testcontainers. Pozwoliło to na testowanie aplikacji w środowisku zbliżonym do produkcyjnego.

Proces testowania obejmował między innymi:

- poprawność uwierzytelniania użytkowników,
- działanie autoryzacji endpointów,
- walidację danych wejściowych,
- obsługę błędów,

-
- poprawność operacji na bazie danych.

3.2 Aplikacja webowa

Aplikacja webowa stanowi główny interfejs użytkownika systemu Wydatkomat. Umożliwia zarządzanie wydatkami, znajomymi, płatnościami oraz profilem użytkownika bez konieczności instalowania dodatkowego oprogramowania.

Frontend został zaimplementowany przy użyciu biblioteki React z wykorzystaniem języka TypeScript oraz narzędzia Vite. Nawigacja pomiędzy widokami realizowana jest przy użyciu biblioteki React Router.

Komunikacja z backendem odbywa się przy użyciu REST API oraz protokołu HTTPS.

3.3 Aplikacja mobilna

Aplikacja mobilna została przygotowana dla systemu Android z wykorzystaniem frameworka React Native oraz środowiska Expo SDK 54. Interfejs aplikacji został zaimplementowany w języku TypeScript.

Do zarządzania nawigacją wykorzystano Expo Router, natomiast komunikacja z backendem realizowana jest przy użyciu biblioteki Axios. Dane uwierzytelniające użytkownika przechowywane są lokalnie przy użyciu Expo Secure Store.

Aplikacja mobilna umożliwia korzystanie z podstawowych funkcji systemu z poziomu urządzeń mobilnych.

3.4 Aplikacja desktopowa

Aplikacja desktopowa została napisana w języku Python i pełni głównie funkcję panelu administracyjnego systemu.

Klient desktopowy umożliwia zarządzanie użytkownikami systemu oraz wykonywanie operacji administracyjnych niedostępnych z poziomu aplikacji webowej i mobilnej.

Komunikacja aplikacji z backendem odbywa się przy użyciu REST API.

4 Podział obowiązków w zespole

W ramach realizacji projektu prace zostały podzielone pomiędzy członków zespołu w sposób umożliwiający równoległe prowadzenie prac nad różnymi obszarami systemu.

W trakcie realizacji projektu prace były prowadzone w sposób iteracyjny z wykorzystaniem tablicy Kanban. Zadania były dzielone na mniejsze elementy i przypisywane do poszczególnych członków zespołu. Pozwoliło to na bieżące monitorowanie postępu prac, kontrolę priorytetów oraz równoległą realizację poszczególnych modułów systemu.

Poniżej przedstawiono zakres odpowiedzialności poszczególnych osób.

4.1 Backend

Za implementację warstwy backendowej odpowiedzialny był Sebastian Górski.

Zakres obowiązków obejmował:

- implementację REST API,
- logikę biznesową systemu,
- integrację z bazą danych,
- implementację mechanizmów autoryzacji i uwierzytelniania.

4.2 Klient web

Za implementację aplikacji webowej odpowiedzialny był Jakub Grzymisławski.

Zakres obowiązków obejmował:

- implementację interfejsu użytkownika w React,
- wykorzystanie TypeScript oraz Vite,
- przygotowanie struktury komponentów aplikacji,
- implementację nawigacji z użyciem React Router,
- integrację z backendem poprzez REST API,
- obsługę stanu aplikacji i komunikacji z API.

4.3 Klient mobile

Za implementację aplikacji mobilnej odpowiedzialny był Rafał Wilczewski.

Zakres obowiązków obejmował:

- implementację aplikacji w React Native,
- wykorzystanie Expo SDK,
- przygotowanie interfejsu użytkownika w TypeScript,
- konfigurację nawigacji z użyciem Expo Router,
- integrację z backendem poprzez REST API,
- obsługę lokalnego przechowywania danych (Expo Secure Store),
- zarządzanie stanem aplikacji.

4.4 Klient desktop

Za implementację aplikacji desktopowej odpowiedzialny był Łukasz Szenkiel.

Zakres obowiązków obejmował:

- implementację aplikacji w języku Python,
- przygotowanie interfejsu użytkownika,
- integrację z backendem poprzez REST API,
- implementację funkcjonalności panelu administracyjnego,
- obsługę podstawowych operacji zarządzania użytkownikami.

4.5 Bezpieczeństwo systemu

Za implementację mechanizmów bezpieczeństwa odpowiedzialny był cały zespół.

Zakres obowiązków obejmował:

- implementację zabezpieczeń systemów we wszystkich modułach,
- skonfigurowanie i opublikowanie API oraz frontendu z wykorzystaniem protokołu HTTPS i TLS 1.3,
- konfigurację 2FA,
- poprawnie zaprogramowaną walidację danych wejściowych użytkownika,
- wdrożenie mechanizmów ochrony przed atakami.

4.6 Baza danych

Za projekt i implementację bazy danych odpowiedzialny był:

- Sebastian Górski

Zakres obowiązków obejmował:

- projekt struktury bazy danych,
- implementację encji,
- konfigurację migracji,
- optymalizację relacji między tabelami.

4.7 Deployment i infrastruktura

Za przygotowanie środowiska uruchomieniowego odpowiedzialny był:

- Sebastian Górski
- Jakub Grzymisławski

Zakres obowiązków obejmował:

- konfigurację Docker oraz Docker Compose,
- wdrożenie aplikacji na serwer VPS,
- konfigurację HTTPS i certyfikatów TLS,
- automatyzację procesu uruchamiania systemu.

4.8 Testy

Za implementację testów odpowiedzialny był cały zespół. Każda osoba była odpowiedzialna za przetestowanie własnego modułu.

5 Instrukcja uruchomienia systemu

Poniżej przedstawiono instrukcję uruchomienia poszczególnych modułów systemu Wydatkomat w środowisku Linux.

5.1 Backend (REST API)

Uruchomienie backendu realizowane jest przy użyciu konteneryzacji Docker.

1. Przejść do katalogu głównego projektu.
2. W przypadku pierwszego uruchomienia skopiować plik konfiguracyjny:

```
cp .env.example .env
```

3. Dostosować konfigurację w pliku `.env` (np. dane do bazy danych, konfiguracja środowiska).
4. Uruchomić aplikację przy użyciu Makefile:

```
make up
```

Po wykonaniu powyższych kroków aplikacja backendowa zostanie uruchomiona wraz z wymaganymi usługami w kontenerach Docker.

5.2 Aplikacja webowa

Aplikacja webowa została przygotowana do uruchomienia w trybie produkcyjnym w środowisku lokalnym.

1. Przejść do katalogu projektu:

```
cd expense-splitter-web
```

-
2. Zainstalować zależności:

```
npm install
```

3. Utworzyć plik konfiguracyjny:

```
cp .env.example .env
```

4. Ustawić adres API w pliku `.env`:

```
VITE_API_URL=https://wydatkomat.tech/api
```

5. Zbudować aplikację:

```
npm run build
```

6. Uruchomić podgląd wersji produkcyjnej:

```
npm run preview
```

Aplikacja dostępna jest następnie pod adresem lokalnym wskazanym przez serwer Vite.

5.3 Aplikacja mobilna

Aplikacja mobilna została przygotowana w technologii React Native z wykorzystaniem Expo.

1. Sklonować repozytorium:

```
git clone https://github.com/Rafal-wq/expense-splitter-mobile.git
```

2. Przejść do katalogu projektu:

```
cd expense-splitter-mobile
```

3. Zainstalować zależności:

```
npm install
```

4. Utworzyć plik konfiguracyjny:

```
cp .env.example .env
```

5. Ustawić adres backendu:

```
EXPO_PUBLIC_API_URL=https://wydatkomat.tech/api
```

6. Uruchomić aplikację:

```
npx expo start
```

Po uruchomieniu należy zeskanować kod QR przy użyciu aplikacji Expo Go.

5.4 Aplikacja desktopowa

Aplikacja desktopowa została napisana w języku Python i nie wymaga procesu budowania.

1. Przejść do katalogu projektu:

```
cd expense-splitter-desktop
```

2. Utworzyć wirtualne środowisko:

```
python3 -m venv venv
```

3. Aktywować środowisko:

```
source venv/bin/activate
```

4. Zainstalować zależności:

```
pip install -r requirements.txt
```

5. Uruchomić aplikację:

```
python main.py
```

Przy pierwszym uruchomieniu aplikacja wyświetla ekran logowania, a następnie inicjalizuje główne okno systemu.

6 Wnioski projektowe

Realizacja projektu Wydatkomat pozwoliła na zaprojektowanie oraz implementację kompletnego systemu wieloplatformowego opartego o architekturę klient–serwer.

W trakcie realizacji projektu istotnym elementem było zapewnienie bezpieczeństwa systemu. Wdrożone mechanizmy obejmowały m.in. uwierzytelnianie JWT, logowanie OAuth2, uwierzytelnianie dwuetapowe, szyfrowanie komunikacji przy użyciu HTTPS oraz TLS, a także stosowanie bezpiecznego przechowywania haseł z wykorzystaniem algorytmu Argon2. Zastosowane rozwiązania pozwoliły na spełnienie wymagań związanych z ochroną danych użytkowników oraz zabezpieczeniem dostępu do systemu.

Istotnym aspektem projektu była również integracja wielu platform klienckich z jednym wspólnym backendem. Wymagało to zaprojektowania spójnego i stabilnego API, które mogło być wykorzystywane przez aplikację webową, mobilną oraz desktopową bez konieczności modyfikacji logiki biznesowej.

Wdrożeni klienci na trzy różne platformy spełniają w pełni swoje zadanie, dzięki czemu doświadczenie użytkownika końcowego jest pozytywne, a dane wprowadzane do REST API jest walidowane dwuetapowo – w warstwie prezentacji oraz logicznej. Dzięki zastosowanym mechanikom użytkownicy mogą korzystać z aplikacji desktopowej oraz mobilnej bez połączenia z siecią WWW.

Podczas realizacji projektu napotkano problemy związane z integracją mechanizmów bezpieczeństwa, zarządzaniem sesją użytkownika oraz synchronizacją danych pomiędzy klientami. Problemy te zostały rozwiązane poprzez iteracyjne podejście do implementacji oraz testowanie poszczególnych komponentów systemu.

Projekt umożliwił zdobycie praktycznej wiedzy z zakresu projektowania systemów rozproszonych, bezpieczeństwa aplikacji webowych oraz integracji wielu technologii w ramach jednego ekosystemu. W przyszłości system może zostać rozbudowany o dodatkowe funkcjonalności, takie jak integracja z systemami płatności, rozszerzone statystyki finansowe lub automatyczne rozliczenia między użytkownikami.

7 Źródła

W ramach realizacji projektu wykorzystano następujące repozytoria oraz zasoby zewnętrzne:

- Repozytorium backendu (REST API):
<https://github.com/sgorski00/expense-splitter-api/>
- Dokumentacja API (OpenAPI / Swagger):
<https://sgorski00.github.io/expense-splitter-api/>
- Repozytorium aplikacji webowej:
https://github.com/jgrzymislowski/aplikacja_webowa
- Repozytorium aplikacji mobilnej:
<https://github.com/Rafal-wq/expense-splitter-mobile>
- Repozytorium aplikacji desktopowej:
<https://github.com/lukaszsz1991/expensemanager-desktop>
- Wdrożona aplikacja produkcyjna:
<https://wydatkomat.tech>